

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序序设计

批改人：

实验	基于 OpenMP 的并行矩阵乘法	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/4/28

1. 实验目的

使用 OpenMP 实现并行通用矩阵乘法，并通过实验分析不同进程数量、矩阵规模、调度机制时该实现的性能。

2. 实验过程和核心代码

任务 1:

OpenMP (Open Multi-Processing) 是一个用于共享内存多处理器编程的并行编程接口。它通过编译指令（如`#pragma omp parallel`）、库函数和环境变量，简化了多线程程序的开发，使程序员可以在串行程序的基础上快速地引入并行性。OpenMP 主要适用于在单机多核环境下实现循环并行、任务并行和数据并行，特点是使用简单、开发成本低、可扩展性好，广泛应用于科学计算、工程仿真等领域。

OpenMP 和 Pthreads 都是实现并行计算的工具，但面向的层次不同。OpenMP 属于高级抽象，通过编译指令自动管理线程的创建、调度和同步，开发者主要关注并行结构本身，编程简单、维护方便；而 Pthreads 属于底层接口，需要程序员手动创建线程、控制同步机制（如互斥锁、条件变量等），灵活性高但开发复杂度大。总体而言，OpenMP 适合快速开发和多核加速，Pthreads 更适合需要精细控制线程行为的复杂并发场景。

OpenMP 在调度时提供了多种模式。Static 模式将迭代次数按块平均划分给线程，划分在程序开始时就完成。适用于各迭代间计算量相近、负载均衡良好的场景。优点是开销低、调度简单，但当每次迭代工作量差异较大时可能造成线程空闲，降低并行效率。

Dynamic 模式线程运行时动态领取任务块，适用于迭代工作量不均衡的场景。能够有效避免线程因任务不均而闲置，提高资源利用率。缺点是调度开销较高。

Auto 模式将调度策略交由编译器或 OpenMP 运行时系统决定。它会综合考虑系统硬件、线程数、负载等因素进行优化选择。适合对性能敏感但又无法预先判断负载模式的程序，但其行为不透明，适合做性能试验或初步测试。

在实现时，接收外部传入的矩阵规模、线程数、调度模式设置，使用 `#pragma omp for` 对矩阵乘法的外循环进行并行化调度。代码如下：

```
// 设置调度策略
if (schedule_type == 1)
    omp_set_schedule(omp_sched_static, 0);
else if (schedule_type == 2)
    omp_set_schedule(omp_sched_dynamic, 0);
else
    omp_set_schedule(omp_sched_auto, 0);

double start_time = omp_get_wtime();

#pragma omp parallel
{
    // 仅线程 0 打印当前调度策略
    if (omp_get_thread_num() == 0) {
        omp_sched_t kind;
        int chunk_size;
        omp_get_schedule(&kind, &chunk_size);
        printf("Using schedule: %s", get_sched_name(kind));
    }

    #pragma omp for schedule(runtime)
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < k; j++) {
            double sum = 0.0;
            for (int l = 0; l < n; l++) {
                sum += A[i * n + l] * B[l * k + j];
            }
            C[i * k + j] = sum;
        }
    }
}
```

输出示例如下：

```
cake@CAK:~/并行程序设计/hw6$ gcc -g -Wall -fopenmp -o mm mm.c
cake@CAK:~/并行程序设计/hw6$ ./mm 1024 1024 1024 4 0
Using schedule: auto
Time taken: 0.678013 seconds
```

任务 2:

首先按照要求实现 `parallel_for` 函数。思路是将循环中的迭代尽量顺序平均分配到指定的线程数的线程中。编译成动态链接库给主程序调用。同时编写了参数传递的数据结构和每次迭代体执行的函数。部分核心代码如下：

`parallel_for` 核心代码如下，目前完成比较简单，只完成了平均分配，类似于 `static` 模式，后续可能在此基础上继续添加：

```
typedef struct {
    int start, end, inc;
    void (*func)(int, void *);
    void *args;
} thread_data;

void *thread_worker(void *arg) {
    thread_data *data = (thread_data *)arg;
    for (int i = data->start; i < data->end; i += data->inc) {
        data->func(i, data->args);
    }
    return NULL;
}
```

```

void parallel_for(int start, int end, int inc,
                 void (*func)(int, void *),
                 void *args, int num_threads) {
    pthread_t threads[num_threads];
    thread_data td[num_threads];

    int total = end - start;
    int chunk = total / num_threads;
    int remainder = total % num_threads;

    int current_start = start;
    for (int i = 0; i < num_threads; i++) {
        int current_end = current_start + chunk + (i < remainder ? 1 : 0);
        td[i].start = current_start;
        td[i].end = current_end;
        td[i].inc = inc;
        td[i].func = func;
        td[i].args = args;

        pthread_create(&threads[i], NULL, thread_worker, &td[i]);
        current_start = current_end;
    }

    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
    }
}

```

主进程中参数结构和迭代实现函数定义如下，在迭代时主要对最外层循环进行并行处理：

```

typedef struct {
    double *A;
    double *B;
    double *C;
    int m, n, k;
} functor_args;

typedef struct {
    void (*func)(int idx, void *args);
    functor_args *args_data;
} functor;

void matmul_row(int idx, void *args) {
    functor_args *fargs = (functor_args *)args;
    int n = fargs->n;
    int k = fargs->k;
    double *A = fargs->A;
    double *B = fargs->B;
    double *C = fargs->C;

    for (int j = 0; j < k; ++j) {
        C[idx * k + j] = 0.0;
        for (int l = 0; l < n; ++l) {
            C[idx * k + j] += A[idx * n + l] * B[l * k + j];
        }
    }
}

```

```

struct timeval start, end;
gettimeofday(&start, NULL);

parallel_for(0, m, 1, f.func, f.args_data, num_threads);

gettimeofday(&end, NULL);

```

3. 实验结果

任务 1:

使用不同的三种模式进行运行得到的结果如下（由于我当前版本不再支持默认模式，所以使用 auto 模式进行代替）

Auto 模式

线程数	矩阵规模				
	128	256	512	1024	2048
1	0.003878	0.032564	0.301764	2.551651	93.886316
2	0.002016	0.016808	0.151060	1.275588	43.554906

4	0.001473	0.008885	0.080845	0.672566	21.634387
8	0.001257	0.007850	0.055057	0.421588	14.102084
16	0.015425	0.013376	0.042667	0.351857	11.654327

Static 模式

线程数	矩阵规模				
	128	256	512	1024	2048
1	0.004098	0.032278	0.312386	2.567914	92.139825
2	0.002048	0.016762	0.150922	1.281296	43.636428
4	0.001486	0.008702	0.082390	0.656609	21.383480
8	0.001176	0.007824	0.052762	0.449085	14.046238
16	0.014337	0.011897	0.047923	0.362224	11.325673

Dynamic 模式

线程数	矩阵规模				
	128	256	512	1024	2048
1	0.003809	0.033109	0.305690	2.663924	92.589172
2	0.002185	0.016433	0.151935	1.382189	44.374328
4	0.001283	0.008769	0.082185	0.675490	20.497048
8	0.000872	0.005178	0.047897	0.419978	13.640522
16	0.012283	0.012076	0.043780	0.372792	8.646019

三种模式内部表现出来的特性与前几次作业分析并行化对矩阵计算影响的一致，在矩阵规模小时线程调度开销会成为性能瓶颈，在矩阵规模大时线程数的增加可以有效降低性能。

观察三种模式的性能表现，发现没有什么特别大的区别，分析应该是矩阵计算时对于任务分配过于均衡，并且本实验限定的矩阵规模并没有达到足以让调度开销成为性能瓶颈的地步。导致差距并不大。

任务 2:

首先测试程序的正确性，使用 2*2 的矩阵乘法进行测试，打印结果如下：

```
● cake@CAK:~/并行程序设计/hw6$ gcc -o main main.c -L. -lparallel -lpthread -Wl,-rpath=.
● cake@CAK:~/并行程序设计/hw6$ ./main 2 2 2 2
Matrix multiplication (A: 2x2, B: 2x2) completed in 0.000300 s
A:
7.30 1.10
9.70 6.20
B:
8.80 4.20
5.90 6.20
C:
70.73 37.48
121.94 79.18
```

可以验证，计算结果是正确的。

接下来测试其效率，结果如下（单位为 s）：

线程数	矩阵规模				
	128	256	512	1024	2048
1	0.006594	0.047344	0.390176	3.631609	111.181976
2	0.003180	0.025560	0.200461	1.805353	54.389311
4	0.002061	0.013012	0.103577	0.933250	26.794992
8	0.001356	0.008540	0.074622	0.575921	17.819695
16	0.001572	0.007880	0.057441	0.457990	13.439779

这次实验最开始我使用的是二维数组和 float 类型，导致运行时间相比于 OpenMP 快了非常多，最后经过调整，统一成 double 类型和一维数组，时间区间变成一致的了。

从比较可以发现，除了在小规模、多进程时手动实现的程序性能占优（推测可能是 OPENMP 的实现存在其他的额外开销），其余的基本都是 OPENMP 的要快，推测是其内部有一些其他的优化方式。对比串行的实现，串行实现的运行效果如下：

```
● cake@CAK:~/并行程序设计/hw6$ ./main_serial 128 128 128
Serial matrix multiplication (A: 128x128, B: 128x128) completed in 5.076 ms
● cake@CAK:~/并行程序设计/hw6$ ./main_serial 256 256 256
Serial matrix multiplication (A: 256x256, B: 256x256) completed in 44.808 ms
● cake@CAK:~/并行程序设计/hw6$ ./main_serial 512 512 512
Serial matrix multiplication (A: 512x512, B: 512x512) completed in 426.966 ms
● cake@CAK:~/并行程序设计/hw6$ ./main_serial 1024 1024 1024
Serial matrix multiplication (A: 1024x1024, B: 1024x1024) completed in 3760.976 ms
● cake@CAK:~/并行程序设计/hw6$ ./main_serial 2048 2048 2048
Serial matrix multiplication (A: 2048x2048, B: 2048x2048) completed in 113724.249 ms
```

对比可以看到，相比于串行时，并行确实可以明显提高性能。

此次实验只是为了粗略证明程序的正确性和效率，更详细的实验分析我会在下一次实验中进行。

4. 实验感想

通过此次实验，我了解到 OpenMp 的使用方式以及其应用方法，并尝试使用 Pthread 库尝试实现 OpenMp 中的 `parallel_for`，深刻体会到 OpenMp 的使用便捷性，并且意外了解了数据结构（多维数组 vs 一维数组）和数据类型对性能的影响。为下一次实验准备了基础。受益匪浅。